

DEVOPS LABORATORY

Project 6: Social Media Underlying Infra Challenges

Demonstrating Kubernetes Cluster Autoscaling for a Social Media Application

Name: Aarya Balwadkar

Roll No / PRN: 23070122005

Class / Division: Devops L1

Date of Submission: 16-8-2026

1. Aim

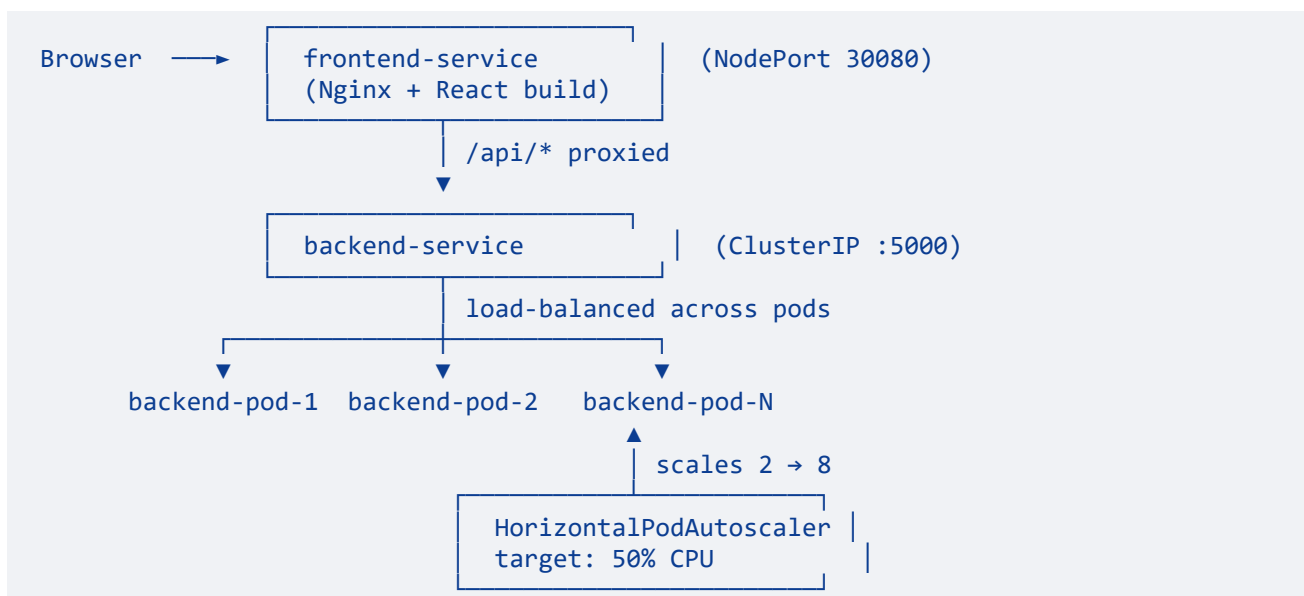
To design and deploy a simple social media web application using a microservice-style architecture (React frontend and Node.js/Express backend), containerize it using Docker, and demonstrate application scalability in Kubernetes by configuring a cluster with the Horizontal Pod Autoscaler (HPA) feature.

2. Objectives

- Build a simple, working social media application (post, view feed, like) split into a frontend and backend service.
- Containerize both services using Docker.
- Set up a local single-node Kubernetes cluster using Minikube.
- Deploy the application to Kubernetes using Deployments and Services.
- Configure a Horizontal Pod Autoscaler (HPA) on the backend service based on CPU utilization.
- Generate artificial load on the application and observe Kubernetes automatically scale the number of backend pods up, then scale back down once load subsides.
- Capture proof of each step (commands + output) for lab submission.

3. System Architecture

The application follows a simple two-tier architecture. The React frontend is built and served as static files through an Nginx container, which also reverse-proxies API calls to the backend. The Node.js/Express backend exposes a REST API and stores posts in memory (no external database is needed for this demo, keeping the project simple). Both tiers run as independent Kubernetes Deployments, each exposed internally via a Kubernetes Service. The backend Deployment has a HorizontalPodAutoscaler attached to it, which is the core feature being demonstrated.



4. Tools & Technology Stack

Tool	Purpose
React 18	Frontend UI — post feed, create post, like button
Node.js + Express	Backend REST API (in-memory data store)
Docker	Containerizing frontend and backend
Nginx	Serves the built React app and proxies API requests
Kubernetes (Minikube)	Local single-node cluster for deployment
kubectl	CLI to interact with the Kubernetes cluster
Metrics Server	Supplies CPU usage data required by the HPA
HorizontalPodAutoscaler (HPA)	The autoscaling feature being demonstrated

5. Prerequisites Verified

The following tools were already installed and verified on the lab machine before starting:

```
C:\Users\Aarya>.kubectl.exe version --client
Client Version: v1.36.0
Kustomize Version: v5.8.1

C:\Users\Aarya>minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
```

NOTE: Docker Desktop must also be installed and running, since Minikube will use Docker as its driver in this guide.

6. Reference: Installing kubectl and Minikube on Windows

NOTE: You confirmed kubectl v1.36.0 and Minikube v1.38.1 are already installed on this machine, so this section is included only as a reference for the lab report. Skip actually re-running these steps and go directly to Section 7.

6.1 Installing kubectl on Windows

Option A — using the official curl-based download (PowerShell):

```
curl.exe -LO "https://dl.k8s.io/release/v1.36.0/bin/windows/amd64/kubectl.exe"
```

Then move kubectl.exe into a folder that is on your PATH (e.g. C:\kubectl\) and verify:

```
kubectl version --client
```

Option B — using Chocolatey (if installed):

```
choco install kubernetes-cli
```

6.2 Installing Minikube on Windows

Download and run the official Windows installer:

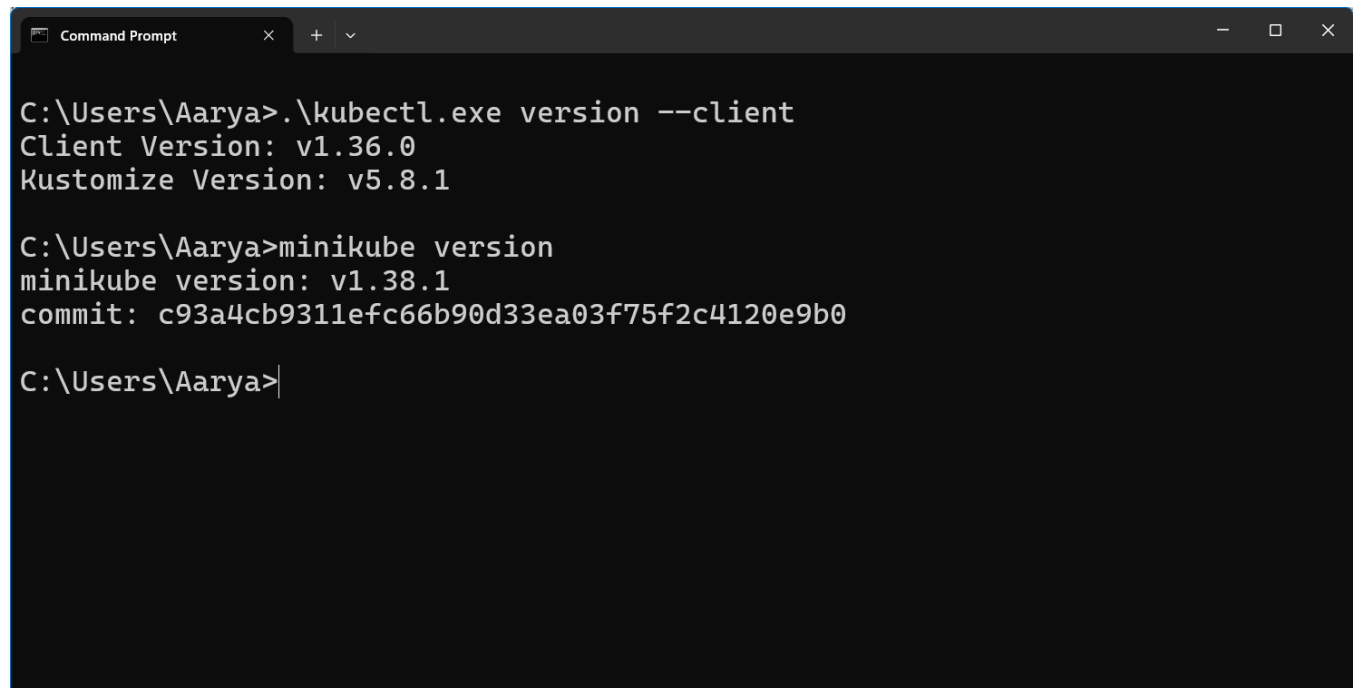
- Go to <https://minikube.sigs.k8s.io/docs/start/>
- Select Windows → x86-64 → .exe installer
- Run minikube-installer.exe and complete the setup wizard
- The installer automatically adds minikube.exe to the system PATH (typically under C:\Program Files\Kubernetes\Minikube\)

Verify the installation in a new terminal:

```
minikube version
```

TIP: Minikube needs a driver to actually run the cluster — a container engine or hypervisor such as Docker, Hyper-V, or VirtualBox. This guide uses Docker as the driver since it is already installed.

6.3 Installed Versions on This Machine (Proof)



```
Command Prompt
C:\Users\Aarya>.\kubectl.exe version --client
Client Version: v1.36.0
Kustomize Version: v5.8.1

C:\Users\Aarya>minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0

C:\Users\Aarya>
```

7. Project Setup

7.1 Folder Structure

Create the following folder structure on your machine (e.g. inside C:\Users\Aarya\social-k8s\):

```
social-k8s/
├── backend/
│   ├── server.js
│   ├── package.json
│   └── Dockerfile
├── frontend/
│   ├── src/App.js
│   ├── src/index.js
│   ├── public/index.html
│   ├── nginx.conf
│   ├── package.json
│   └── Dockerfile
└── k8s/
    ├── backend-deployment.yaml
    ├── frontend-deployment.yaml
    └── backend-hpa.yaml
```

TIP: All these files are provided in the social-k8s.zip project package. Extract it and open the folder in your terminal / VS Code — you do not need to type the code out manually.

7.2 Backend — server.js (Express API)

A minimal REST API with an in-memory list of posts, plus a /api/stress endpoint used later to generate CPU load for the autoscaling demo:

```
const express = require('express');
const cors = require('cors');
const os = require('os');

const app = express();
app.use(cors());
app.use(express.json());

let posts = [
  { id: 1, author: 'Asha', text: 'Just deployed my first K8s cluster!', likes: 3 },
  { id: 2, author: 'Rohan', text: 'Autoscaling is magic.', likes: 5 }
];
let nextId = 3;

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', pod: os.hostname() });
});

app.get('/api/posts', (req, res) => {
  res.json({ posts, servedBy: os.hostname() });
});
```

```

app.post('/api/posts', (req, res) => {
  const { author, text } = req.body;
  const post = { id: nextId++, author, text, likes: 0 };
  posts.unshift(post);
  res.status(201).json(post);
});

app.post('/api/posts/:id/like', (req, res) => {
  const post = posts.find(p => p.id === parseInt(req.params.id));
  post.likes += 1;
  res.json(post);
});

// CPU-intensive endpoint used to trigger autoscaling
app.get('/api/stress', (req, res) => {
  const start = Date.now();
  let x = 0;
  while (Date.now() - start < 200) { x += Math.sqrt(Math.random()); }
  res.json({ result: x, pod: os.hostname() });
});

app.listen(process.env.PORT || 5000);

```

7.3 Backend — Dockerfile

```

FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install --production
COPY server.js .
EXPOSE 5000
CMD ["node", "server.js"]

```

7.4 Frontend — App.js (React)

The React app displays a feed, a form to create posts, a like button, and shows which backend pod served the last request (useful to visually prove load balancing across replicas). It also includes a 'Run Load Test' button that fires 200 parallel requests at the CPU-heavy /api/stress endpoint — this is the trigger button for your autoscaling demo.

NOTE: Full App.js source is in the provided ZIP under frontend/src/App.js — it is too long to usefully retype here, so reference the file directly in your submission if your instructor wants source code excerpts.

7.5 Frontend — nginx.conf

Nginx serves the built React files and forwards any /api/ request to the backend Kubernetes Service by its internal DNS name:

```
server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        try_files $uri /index.html;
    }
    location /api/ {
        proxy_pass http://backend-service:5000/api/;
    }
}
```

7.6 Frontend — Dockerfile (multi-stage build)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

8. Kubernetes Manifest Files

8.1 backend-deployment.yaml

Defines the backend Deployment (starts with 2 replica pods) and a ClusterIP Service to expose it internally. Note the `resources.requests.cpu` field — this is mandatory, because the HPA calculates CPU percentage relative to this requested value.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
spec:
  replicas: 2
  selector:
    matchLabels: { app: backend }
  template:
    metadata:
      labels: { app: backend }
    spec:
      containers:
        - name: backend
          image: social-backend:latest
          imagePullPolicy: IfNotPresent
          ports: [{ containerPort: 5000 }]
          resources:
            requests: { cpu: "100m", memory: "64Mi" }
```

```

        limits:  { cpu: "200m", memory: "128Mi" }
        readinessProbe:
          httpGet: { path: /api/health, port: 5000 }
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector: { app: backend }
  ports: [{ port: 5000, targetPort: 5000 }]
  type: ClusterIP

```

8.2 frontend-deployment.yaml

Defines the frontend Deployment (1 replica) and a NodePort Service so it is reachable from the browser via Minikube.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
spec:
  replicas: 1
  selector:
    matchLabels: { app: frontend }
  template:
    metadata:
      labels: { app: frontend }
    spec:
      containers:
        - name: frontend
          image: social-frontend:latest
          imagePullPolicy: IfNotPresent
          ports: [{ containerPort: 80 }]
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector: { app: frontend }
  ports: [{ port: 80, targetPort: 80, nodePort: 30080 }]
  type: NodePort

```

8.3 backend-hpa.yaml — the core autoscaling feature

This is the file that fulfills the assignment requirement. It attaches a HorizontalPodAutoscaler to backend-deployment: it keeps a minimum of 2 pods for availability, allows scaling up to a maximum of 8 pods, and targets 50% average CPU utilization as the scaling trigger.

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

```

```

metadata:
  name: backend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend-deployment
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0

```

How this works

- Every backend pod requests 100m CPU. This becomes the 100% baseline for that pod.
- The Metrics Server (enabled in Minikube) reports actual CPU usage of each pod roughly every 15 seconds.
- The HPA controller compares average CPU usage across all backend pods against the 50% target.
- If average usage exceeds 50%, the controller increases the replicas field on backend-deployment — Kubernetes then schedules new pods automatically.
- If usage drops back down, the HPA scales down again, but waits 60 seconds (stabilizationWindowSeconds) before doing so, to avoid rapidly flapping pods up and down.
- Replica count is always kept between minReplicas (2) and maxReplicas (8).

9. Step-by-Step Execution

All commands below are run in Windows PowerShell or Command Prompt. Navigate to the social-k8s folder first (e.g. `cd C:\Users\Aarya\social-k8s`) before starting.

Step 1: Start Minikube

Start the local Kubernetes cluster using Docker as the driver:

```
minikube start --driver=docker
```

```

C:\Users\Aarya>minikube start --driver=docker
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on user configuration
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Downloading Kubernetes v1.35.1 preload ...
  > preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 1.31 Mi
  > gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 1.74 Mi
* Creating docker container (CPUs=2, Memory=3072MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

```

Wait for it to finish provisioning the node, downloading images, and configuring kubectl. Then verify the cluster is up:

```

kubectl get nodes
minikube status

```

```

> preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 1.31 Mi
> gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 1.74 Mi
* Creating docker container (CPUs=2, Memory=3072MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

C:\Users\Aarya>kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  58s   v1.35.1

C:\Users\Aarya>minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

C:\Users\Aarya>

```

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, MCP Toolkit (BETA), and Extensions. The main area displays 'Containers' with a search bar and a toggle for 'Only show running containers'. Below this is a table of containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk rea	Actions
☐	minikube	b0d1db514d42	k8s-miniku	55677:22 Show all ports (5)	23.63%	717.8MB / 3GB	23.37%	0B / 0B	

At the bottom, there's a status bar showing 'Engine running' and a warning: 'Kubernetes failed to start RAM 3.26 GB CPU 5.53% Disk: 17.95 GB used (limit 1006.85 GB)'. A 'Terminal' button is visible on the right.

Step 2: Enable the Metrics Server Addon

The Metrics Server is required for the HPA to read live CPU usage from pods. Without it, kubectl get hpa will show <unknown> for CPU targets.

```
minikube addons enable metrics-server
```

```
C:\Users\Aarya>minikube addons enable metrics-server
* metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub
.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
- Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
* The 'metrics-server' addon is enabled

C:\Users\Aarya>
```

Verify it's running (it may take ~30-60 seconds to become Ready):

```
kubectl get pods -n kube-system | findstr metrics-server
```

```
C:\Users\Aarya>kubectl get pods -n kube-system | findstr metrics-server
metrics-server-9d74bb658-9wgwm      0/1      Running    0      46s

C:\Users\Aarya>
```

Step 3: Point Docker CLI at Minikube's Internal Docker Daemon

This makes 'docker build' place images directly inside the Minikube cluster's image store, so Kubernetes can find them without needing a registry push. Run this in every new terminal window before building images.

```
minikube -p minikube docker-env -shell cmd
```

NOTE: The command above is for cmd. If using Command Prompt (cmd.exe) instead, run: minikube =p minikube docker-env | Invoke-Expression

```
C:\Users\Aarya>minikube -p minikube docker-env --shell cmd
SET DOCKER_TLS_VERIFY=1
SET DOCKER_HOST=tcp://127.0.0.1:55678
SET DOCKER_CERT_PATH=C:\Users\Aarya\.minikube\certs
SET MINIKUBE_ACTIVE_DOCKERD=minikube
REM To point your shell to minikube's docker-daemon, run:
REM @FOR /f "tokens=*" %i IN ('minikube -p minikube docker-env --shell cmd') DO @%i

C:\Users\Aarya>
```

Step 4: Build the Docker Images

With the Minikube docker-env still active in this terminal, build both images:

```
docker build -t social-backend:latest ./backend
docker build -t social-frontend:latest ./frontend
```

Confirm both images exist inside Minikube's Docker:

```
docker images | findstr social
```

```

C:\Users\Aarya>f:

F:\>cd SEM_7

F:\SEM_7>cd Devops_Lab

F:\SEM_7\Devops_Lab>cd social-k8s

F:\SEM_7\Devops_Lab\social-k8s>docker build -t social-backend:latest ./backend
[+] Building 41.1s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 181B                                0.1s
=> [internal] load metadata for docker.io/library/node:20-alpine   4.1s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50 21.1s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c 0.0s
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3 1.26MB / 1.26MB 0.8s
=> => sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea 43.23MB / 43.23MB 16.9s
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 445B / 445B 1.0s
=> => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 3.8s
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 0.2s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 0.1s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 0.1s
=> [internal] load build context                                    0.4s
=> => transferring context: 2.13kB                                   0.3s
=> [2/5] WORKDIR /app                                             2.1s
=> [3/5] COPY package.json .                                     0.1s
=> [4/5] RUN npm install --production                             10.4s
=> [5/5] COPY server.js .                                         0.2s
=> exporting to image                                             2.2s
=> => exporting layers                                              0.9s
=> => exporting manifest sha256:d7246b1b42e0d8571cc8843b861750045c7c18f9cc4437b470e3e6c111 0.0s
=> => exporting config sha256:4c9f432d74f4a773ec1f1893a32b02c782629c5f0bf0256a3f8bdfef53ff 0.1s
=> => exporting attestation manifest sha256:2a691b40ccb9d6dac2d480ac8dc00f9a0a6df9c002aa8c 0.1s
=> => exporting manifest list sha256:57f18f79829b393fbaad5fca3d34143d8a91ae880762a88b7b0cb 0.0s
=> => naming to docker.io/library/social-backend:latest          0.0s
=> => unpacking to docker.io/library/social-backend:latest       0.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/nfk7rsgh3fuil2wyy950kriea

F:\SEM_7\Devops_Lab\social-k8s>docker build -t social-frontend:latest ./frontend
[+] Building 227.4s (16/16) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 330B                                0.1s
=> [internal] load metadata for docker.io/library/nginx:alpine    3.6s
=> [internal] load metadata for docker.io/library/node:20-alpine  1.1s

```

```

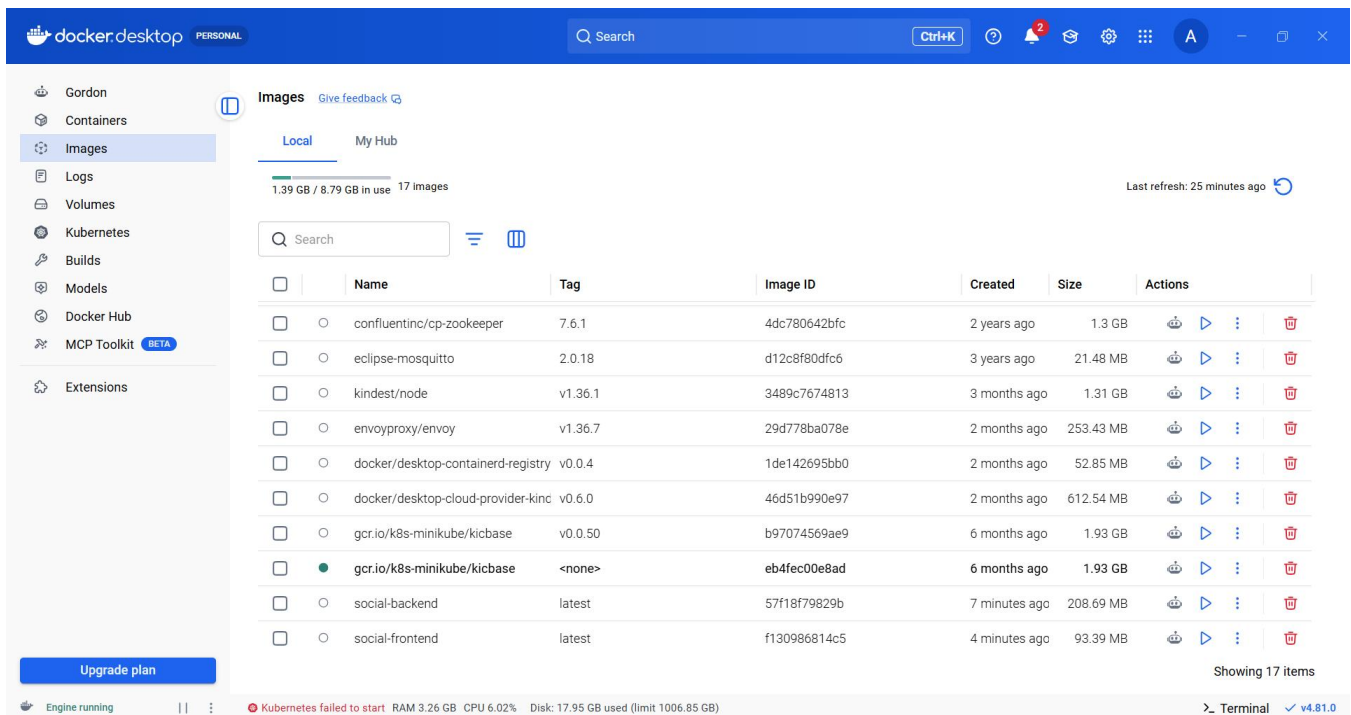
Command Prompt
=> [internal] load metadata for docker.io/library/node:20-alpine 1.1s
=> [auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [build 1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d 0.1s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c 0.1s
=> [internal] load build context 0.7s
=> => transferring context: 6.71kB 0.6s
=> [stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da03789 12.0s
=> => resolve docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6 0.2s
=> => sha256:46519e7231d2eb5604df229beb44d59719a489eaa7aca52982535a010b0 20.31MB / 20.31MB 8.4s
=> => sha256:390dc935348d8070e695fbaae2a4bb114fb9e69c59f628e7576036ee9d524 1.40kB / 1.40kB 1.3s
=> => sha256:d0008c891db48b5f526d914bce9e8d889fe1a9d1f08291ae03fe97f871726 1.21kB / 1.21kB 1.4s
=> => sha256:46f977ee452f4399c208714afa034868d6056864f8a0cf3c643ab143dd802c80 404B / 404B 1.3s
=> => sha256:62bec68d7c31c4c8a19d812d84da5f7748e54690c037979945b6c5b6c924b142 957B / 957B 0.5s
=> => sha256:1223f016b4e4a2c21f7c49d4837fbfd47a9da6436b511690ca1e582fc2810d59 627B / 627B 0.5s
=> => sha256:55afalecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af2 3.85MB / 3.85MB 5.5s
=> => sha256:3cd534fe98c64d68a1f4f1c83abb8d5cba7ecfd7be88e592389929d12e625 1.89MB / 1.89MB 4.2s
=> => extracting sha256:55afalecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 1.0s
=> => extracting sha256:3cd534fe98c64d68a1f4f1c83abb8d5cba7ecfd7be88e592389929d12e6253da 1.0s
=> => extracting sha256:1223f016b4e4a2c21f7c49d4837fbfd47a9da6436b511690ca1e582fc2810d59 0.0s
=> => extracting sha256:62bec68d7c31c4c8a19d812d84da5f7748e54690c037979945b6c5b6c924b142 0.0s
=> => extracting sha256:46f977ee452f4399c208714afa034868d6056864f8a0cf3c643ab143dd802c80 0.0s
=> => extracting sha256:d0008c891db48b5f526d914bce9e8d889fe1a9d1f08291ae03fe97f871726f38 0.0s
=> => extracting sha256:390dc935348d8070e695fbaae2a4bb114fb9e69c59f628e7576036ee9d5244c9 0.1s

Command Prompt
=> => extracting sha256:46519e7231d2eb5604df229beb44d59719a489eaa7aca52982535a010b07a9ed 2.1s
=> CACHED [build 2/6] WORKDIR /app 0.0s
=> [build 3/6] COPY package.json . 0.6s
=> [build 4/6] RUN npm install 205.6s
=> [build 5/6] COPY . . 1.0s
=> [build 6/6] RUN npm run build 13.1s
=> [stage-1 2/3] COPY --from=build /app/build /usr/share/nginx/html 0.1s
=> [stage-1 3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf 0.1s
=> exporting to image 0.6s
=> => exporting layers 0.2s
=> => exporting manifest sha256:75f0330077a414646b2f3c43497031083d8523846db8f302380a0957e8 0.0s
=> => exporting config sha256:a54e13be03680a08cebdec9bf27e8c3ca43ea14447ce50362f28d100e474 0.0s
=> => exporting attestation manifest sha256:7e444a7020b121a74b9e2d3c695d267bab1ee501f4d423 0.0s
=> => exporting manifest list sha256:f130986814c5d89f7b234d1b6a051b34ff840fe24b9879fbf5b56 0.0s
=> => naming to docker.io/library/social-frontend:latest 0.0s
=> => unpacking to docker.io/library/social-frontend:latest 0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/uxpifcuztjgyf3t1m432w9i2p

F:\SEM_7\Devops_Lab\social-k8s>docker images | findstr social
social-backend:latest
57f18f79829b 209MB 50.8MB
social-frontend:latest
f130986814c5 93.4MB 26.2MB

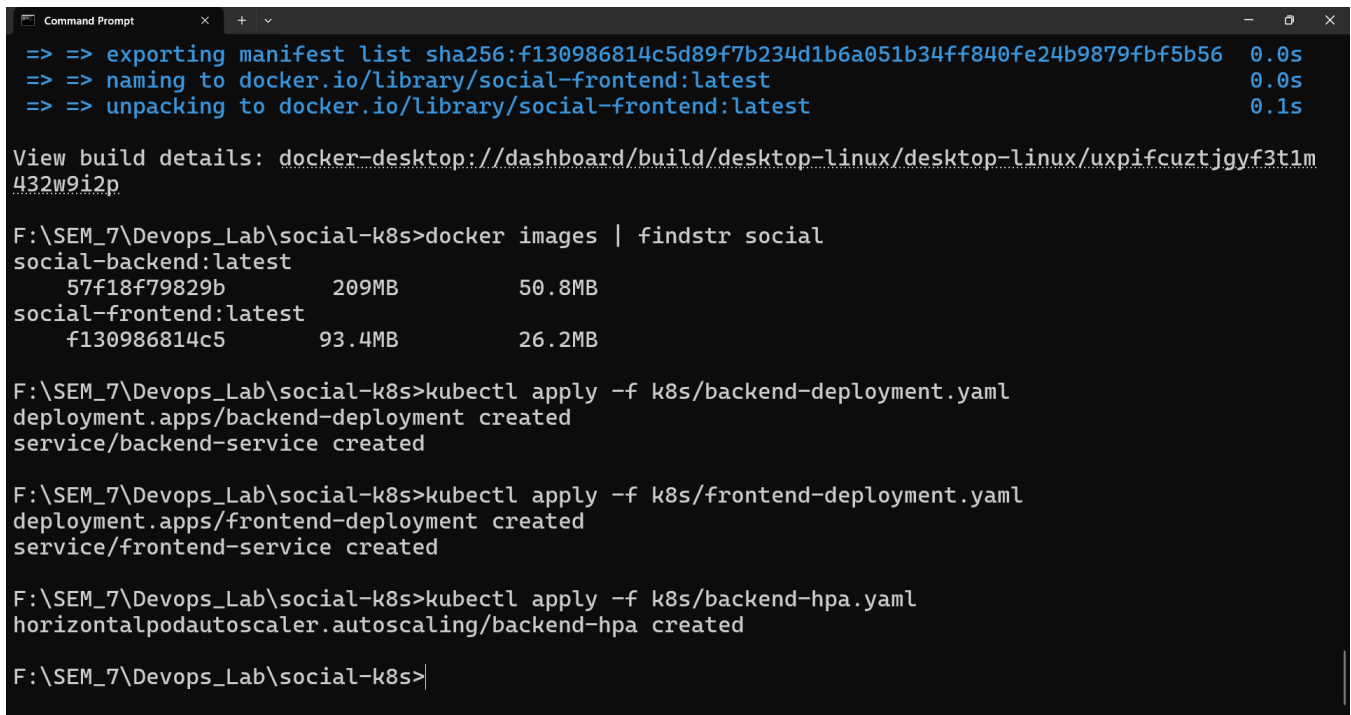
```



Step 5: Deploy the Application to Kubernetes

Apply the backend Deployment + Service, frontend Deployment + Service, and the HPA:

```
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/frontend-deployment.yaml
kubectl apply -f k8s/backend-hpa.yaml
```



Step 6: Verify the Deployment

```
kubectl get pods
kubectl get deployments
kubectl get svc
kubectl get hpa
```

You should see 2 backend pods and 1 frontend pod in Running state, both Services listed, and the HPA showing TARGETS such as 0%/50% (CPU usage will be near 0% since there is no load yet).

```

F:\SEM_7\Devops_Lab\social-k8s>kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
backend-deployment-66cb79749-j4dts  0/1     ErrImagePull        0           38s
backend-deployment-66cb79749-sb279  0/1     ImagePullBackOff    0           38s
frontend-deployment-74b9f7fc98-v564m 0/1     ImagePullBackOff    0           38s

F:\SEM_7\Devops_Lab\social-k8s>kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
backend-deployment  0/2     2             0           38s
frontend-deployment 0/1     1             0           38s

F:\SEM_7\Devops_Lab\social-k8s>kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
backend-service     ClusterIP   10.98.215.229 <none>        5000/TCP         38s
frontend-service    NodePort    10.106.146.65 <none>        80:30080/TCP     38s
kubernetes          ClusterIP   10.96.0.1     <none>        443/TCP          17m

F:\SEM_7\Devops_Lab\social-k8s>kubectl get hpa
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS
AGE
backend-hpa         Deployment/backend-deployment  cpu: <unknown>/50%  2         8         2
37s

F:\SEM_7\Devops_Lab\social-k8s>

```

```

F:\SEM_7\Devops_Lab\social-k8s>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-deployment-66cb79749-26x7f  1/1     Running   0           41s
backend-deployment-66cb79749-bjc8k  1/1     Running   0           41s
frontend-deployment-74b9f7fc98-slvs  1/1     Running   0           41s

F:\SEM_7\Devops_Lab\social-k8s>

```

Step 7: Access the Application in the Browser

```
minikube service frontend-service --url
```

This prints a URL such as `http://127.0.0.1:xxxxx` (Minikube on the Docker driver uses a local tunnel). Open it in your browser — you should see the social feed, the post form, and the 'Run Load Test' button. Note the 'Currently served by pod' banner at the top; refresh a few times and you may see it alternate between backend-deployment-xxxx pod names, proving requests are load-balanced across replicas.

K8s Social

A tiny social feed for demonstrating Kubernetes autoscaling

Currently served by pod: `backend-deployment-66cb79749-26x7f`

Your name

What's happening?

Post

Run Load Test (trigger autoscaling)

Asha

Just deployed my first Kubernetes cluster! 🎉

3

Rohan

Autoscaling is magic. Pods appearing out of nowhere.

5

Meera

DevOps lab going well so far.

2

10. Demonstrating Autoscaling (Core Feature)

This is the main deliverable of the project: proving that Kubernetes automatically increases the number of backend pods under load, and scales back down once load subsides. Open three separate terminal windows, all inside the social-k8s folder.

Step 8: Open Three Terminals for Live Monitoring

Terminal 1 — Watch the HPA live

```
kubectl get hpa -w
```

Leave this running. The -w (watch) flag streams updates as the REPLICAS and CPU columns change.

Terminal 2 — Watch pods scale live

```
kubectl get pods -w
```

Leave this running too. New backend pods will appear here as the HPA scales up.

Terminal 3 — Generate load

Choose ONE of the following two options:

Option A — using a temporary busybox pod inside the cluster to hammer the /api/stress endpoint:

```
kubectl run load-generator --image=busybox:1.36 --restart=Never -it --rm -- /bin/sh -c
"while true; do wget -q -O- http://backend-service:5000/api/stress; done"
```

Option B — open the app in the browser and click the "Run Load Test" button several times in a row (it fires 200 parallel requests each click).

Step 9: Observe the Scale-Up

Within roughly 30-60 seconds of sustained load, you should observe the following in your terminals:

- Terminal 1 (HPA): the CPU percentage column climbs above the 50% target, e.g. 78%/50%.
- Terminal 1 (HPA): shortly after, the REPLICAS column increases, e.g. 2 to 4 to 6.
- Terminal 2 (pods): new pods with names like backend-deployment-xxxxxxx-yyyyy appear and transition from Pending to ContainerCreating to Running.

```
F:\SEM_7\Devops_Lab\social-k8s>kubectl get hpa -w
NAME          REFERENCE                TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa    Deployment/backend-deployment  cpu: 2%/50%    2         8         2         18m
backend-hpa    Deployment/backend-deployment  cpu: 12%/50%   2         8         2         18m
backend-hpa    Deployment/backend-deployment  cpu: 116%/50%  2         8         2         19m
backend-hpa    Deployment/backend-deployment  cpu: 116%/50%  2         8         5         20m
```

```

PS C:\Users\Aarya> kubectl get pods -w
NAME                                READY    STATUS      RESTARTS   AGE
backend-deployment-66cb79749-26x7f  1/1      Running     0           3m
backend-deployment-66cb79749-bjc8k  1/1      Running     0           3m
frontend-deployment-74b9f7fc98-slvvs 1/1      Running     0           3m
load-generator                       0/1      Pending     0           0s
load-generator                       0/1      Pending     0           0s
load-generator                       0/1      ContainerCreating 0           0s
load-generator                       1/1      Running     0           12s
backend-deployment-66cb79749-jv47g  0/1      Pending     0           1s
backend-deployment-66cb79749-ztwk6  0/1      Pending     0           0s
backend-deployment-66cb79749-jfv28  0/1      Pending     0           0s
backend-deployment-66cb79749-jv47g  0/1      Pending     0           1s
backend-deployment-66cb79749-ztwk6  0/1      Pending     0           0s
backend-deployment-66cb79749-jfv28  0/1      Pending     0           0s
backend-deployment-66cb79749-jv47g  0/1      ContainerCreating 0           1s
backend-deployment-66cb79749-jfv28  0/1      ContainerCreating 0           0s
backend-deployment-66cb79749-ztwk6  0/1      ContainerCreating 0           0s
backend-deployment-66cb79749-jv47g  0/1      Running     0           5s
backend-deployment-66cb79749-ztwk6  0/1      Running     0           4s
backend-deployment-66cb79749-jfv28  0/1      Running     0           5s
backend-deployment-66cb79749-jv47g  1/1      Running     0           13s
backend-deployment-66cb79749-jfv28  1/1      Running     0           15s
backend-deployment-66cb79749-ztwk6  1/1      Running     0           16s

```

Step 10: Confirm with a Snapshot

In a fourth terminal (or after stopping the watch commands with Ctrl+C), take a clean snapshot:

```

kubectl get hpa
kubectl get pods -l app=backend

```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Aarya> kubectl get hpa
NAME      REFERENCE                                TARGETS      MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa Deployment/backend-deployment  cpu: 116%/50%    2          8          5          20m
PS C:\Users\Aarya> kubectl get pods -l app=backend
NAME                                READY    STATUS      RESTARTS   AGE
backend-deployment-66cb79749-26x7f  1/1      Running     0           5m4s
backend-deployment-66cb79749-bjc8k  1/1      Running     0           5m4s
backend-deployment-66cb79749-jfv28  1/1      Running     0           27s
backend-deployment-66cb79749-jv47g  1/1      Running     0           28s
backend-deployment-66cb79749-ztwk6  1/1      Running     0           27s
PS C:\Users\Aarya>

```

Step 11: Stop the Load and Observe Scale-Down

Stop the load generator (press Ctrl+C in Terminal 3, or if it doesn't exit cleanly, delete the pod manually):

```

kubectl delete pod load-generator

```

Watch Terminal 1 again. CPU usage will drop back toward 0%, and after the 60-second stabilization window configured in backend-hpa.yaml, the HPA will scale REPLICAS back down toward the minimum of 2.

```

Command Prompt - kubectl g x Windows PowerShell x Windows PowerShell x Windows PowerShell x + v
backend-deployment-66cb79749-26x7f 1/1 Running 0 30s
backend-deployment-66cb79749-bjc8k 1/1 Running 0 30s
backend-deployment-66cb79749-p8b2t 1/1 Terminating 0 8m43s
backend-deployment-66cb79749-xsr5q 1/1 Terminating 0 8m43s
frontend-deployment-74b9f7fc98-slvvs 1/1 Running 0 30s

F:\SEM_7\Devops_Lab\social-k8s>kubectl get pods
NAME READY STATUS RESTARTS AGE
backend-deployment-66cb79749-26x7f 1/1 Running 0 41s
backend-deployment-66cb79749-bjc8k 1/1 Running 0 41s
frontend-deployment-74b9f7fc98-slvvs 1/1 Running 0 41s

F:\SEM_7\Devops_Lab\social-k8s>minikube service frontend-service --url
http://127.0.0.1:52457
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

F:\SEM_7\Devops_Lab\social-k8s>kubectl get hpa -w
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
backend-hpa Deployment/backend-deployment cpu: 2%/50% 2 8 2 18m
backend-hpa Deployment/backend-deployment cpu: 12%/50% 2 8 2 18m
backend-hpa Deployment/backend-deployment cpu: 116%/50% 2 8 2 19m
backend-hpa Deployment/backend-deployment cpu: 116%/50% 2 8 5 20m
backend-hpa Deployment/backend-deployment cpu: 68%/50% 2 8 5 20m
backend-hpa Deployment/backend-deployment cpu: 51%/50% 2 8 5 21m

```

Step 12: Describe the HPA (Optional but Recommended for Viva)

This command shows a detailed event history of every scaling decision the HPA made, which is excellent supporting evidence:

```
kubectl describe hpa backend-hpa
```

Look at the Events section at the bottom of the output. It will show entries like 'New size: 4; reason: cpu resource utilization above target' and later 'New size: 2; reason: All metrics below target'.

```

PS C:\Users\Aarya> kubectl describe hpa backend-hpa
Name: backend-hpa
Namespace: default
Labels: <none>
Annotations: <none>
CreationTimestamp: Sun, 16 Aug 2026 15:37:52 +0530
Reference: Deployment/backend-deployment
Metrics: ( current / target )
  resource cpu on pods (as a percentage of request): 14% (14m) / 50%
Min replicas: 2
Max replicas: 8
Behavior:
  Scale Up:
    Stabilization Window: 0 seconds
    Select Policy: Max
    Policies:
      - Type: Pods Value: 4 Period: 15 seconds
      - Type: Percent Value: 100 Period: 15 seconds
  Scale Down:
    Stabilization Window: 60 seconds
    Select Policy: Max
    Policies:
      - Type: Percent Value: 100 Period: 15 seconds
Deployment pods: 5 current / 5 desired
Conditions:
  Type Status Reason Message
  ----
  AbleToScale True ScaleDownStabilized recent recommendations were higher than current one, applying the highest recent recommendation

```

```

- Type: Pods      Value: 4      Period: 15 seconds
- Type: Percent   Value: 100    Period: 15 seconds
Scale Down:
  Stabilization Window: 60 seconds
  Select Policy: Max
  Policies:
    - Type: Percent Value: 100 Period: 15 seconds
Deployment pods:      5 current / 5 desired
Conditions:
  Type      Status Reason      Message
  ----      -
  AbleToScale True    ScaleDownStabilized recent recommendations were higher than current one, applying the highest recent recommendation
  ScalingActive True    ValidMetricFound    the HPA was able to successfully calculate a replica count from cpu resource utilization (percentage of request)
  ScalingLimited False   DesiredWithinRange  the desired count is within the acceptable range
Events:
  Type      Reason      Age      From      Message
  ----      -
  Warning    FailedComputeMetricsReplicas 20m (x12 over 23m) horizontal-pod-autoscaler invalid metrics (1 invalid out of 1), first error is: failed to get cpu resource metric value: failed to get cpu utilization: unable to get metrics for resource cpu: no metrics returned from resource metrics API
  Warning    FailedGetResourceMetric 13m (x41 over 23m) horizontal-pod-autoscaler failed to get cpu utilization: unable to get metrics for resource cpu: no metrics returned from resource metrics API
  Warning    FailedGetResourceMetric 7m47s horizontal-pod-autoscaler failed to get cpu utilization: did not receive metrics for targeted pods (pods might be unready)
  Normal     SuccessfulRescale 3m29s horizontal-pod-autoscaler New size: 5; reason: cpu resource utilization (percentage of request) above target
PS C:\Users\Aarya>

```

13. Conclusion

This project successfully demonstrates Kubernetes' application scalability feature using a simple social media application as the workload. A React frontend and Node.js backend were containerized with Docker and deployed onto a local Minikube cluster using standard Kubernetes Deployment and Service objects. A HorizontalPodAutoscaler was configured on the backend service, targeting 50% average CPU utilization with a replica range of 2 to 8 pods. Under artificially generated load, Kubernetes automatically detected the increased CPU usage via the Metrics Server and scaled the number of backend pods upward in real time, distributing incoming traffic across the new replicas. Once the load was removed, the cluster automatically scaled back down to the minimum replica count after a stabilization period. This confirms that Kubernetes can elastically and automatically manage compute resources in response to real-time demand, which is a foundational capability for running resilient, cost-efficient infrastructure for social media and other high-traffic applications.